Universitat Oberta de Catalunya

Máster Universitario en Diseño y Programación de Videojuegos

M7.459 - Inteligencia artificial para videojuegos

PEC 1 - IA para juegos en primera persona

Alvar Dalda Svärd

Curso 2026/27

2 de noviembre de 2026

Índice

1. Introducción **2**

2. Actividad 1: escenario y NavMesh **3**

2.1. Descripción del escenario 3

2.2. Configuración de la NavMesh 3

2.3. Interfaz de usuario 3

2.4. Evidencias visuales 4

3. Actividad 2: agente fantasma (*wander*) **5**

3.1. Objetivo y restricciones 5

3.2. Algoritmo 5

3.3. Implementación 5

3.4. Vídeo 6

4. Actividad 3: agente de seguridad (máquina de estados) **7**

4.1. Objetivo 7

4.2. Esquema de la máquina de estados 7

4.3. Estado Patroll 7

4.4. Estado Flee 8

4.5. Transiciones 8

4.6. Animación de caminar 8

4.7. Evidencias visuales 8

4.8. Vídeo 10

5. Controles de demo **11**

6. Recursos ajenos **11**

7. IA generativa **12**

7.1. Corrección de la lógica de huida del vigilante 12

7.2. Creación del script de cámara (`CameraPOV`) 12

7.3. Animación de caminar del Knight 13

8. Conclusiones **14**

1. Introducción

Esta PEC es la primera de tres entregas encadenadas: la PEC2 partirá de la PEC1 y la PEC3 de la PEC2. El tema es un simulador de sala de castillo o jardín encantado con agentes autónomos.

En esta primera entrega se ha hecho:

- Un escenario 3D con obstáculos no caminables y *NavMesh*.
- Un fantasma con *wander* solo en Visual Scripting.
- Un vigilante (Knight) con máquina de estados (patrulla y huida), también en Visual Scripting.

Me he apoyado en el material de la asignatura [1] y en el artículo de Reynolds sobre *steering behaviors* [2]. Todo está hecho en Unity (URP) con AI Navigation [3] y Visual Scripting [4].

La entrega es un ZIP con este PDF, dos vídeos (actividades 2 y 3) y un `.txt` con el enlace al repositorio [5]: https://github.com/Raptor200111/IAV_R1. El PDF y los vídeos también están en ese repo, junto con el proyecto.

2. Actividad 1: escenario y NavMesh

2.1. Descripción del escenario

La escena es `InnerCastleGardenScene`: un patio cerrado por un muro (`OutterWall`), suelo caminable (`Ground_03`) y decoración low-poly (árboles, arbustos, picnic, etc.). Lo que debe cortar la NavMesh cuelga de un padre `Obstacles`; hierba y flores se dejan fuera para no llenar la malla de agujeros inútiles.

He dejado hueco libre en el patio para los agentes de las PEC siguientes.

2.2. Configuración de la NavMesh

Según el material del curso [1]:

- **NavMesh Surface** en el suelo, área *Walkable*, geometría con *Physics Colliders*, recolección *All Game Objects*.
- **NavMesh Modifier** en `Obstacles` (*Apply To Children*, *Not Walkable*) para muros, árboles y picnic.
- Bake de la superficie. En props estáticos no uso `NavMesh Obstacle` (eso va para obstáculos que se mueven).

2.3. Interfaz de usuario

En pantalla (Canvas / TextMeshPro) aparece el identificador que pide el enunciado: `Alvar Dalda @ 2026`.

2.4. Evidencias visuales

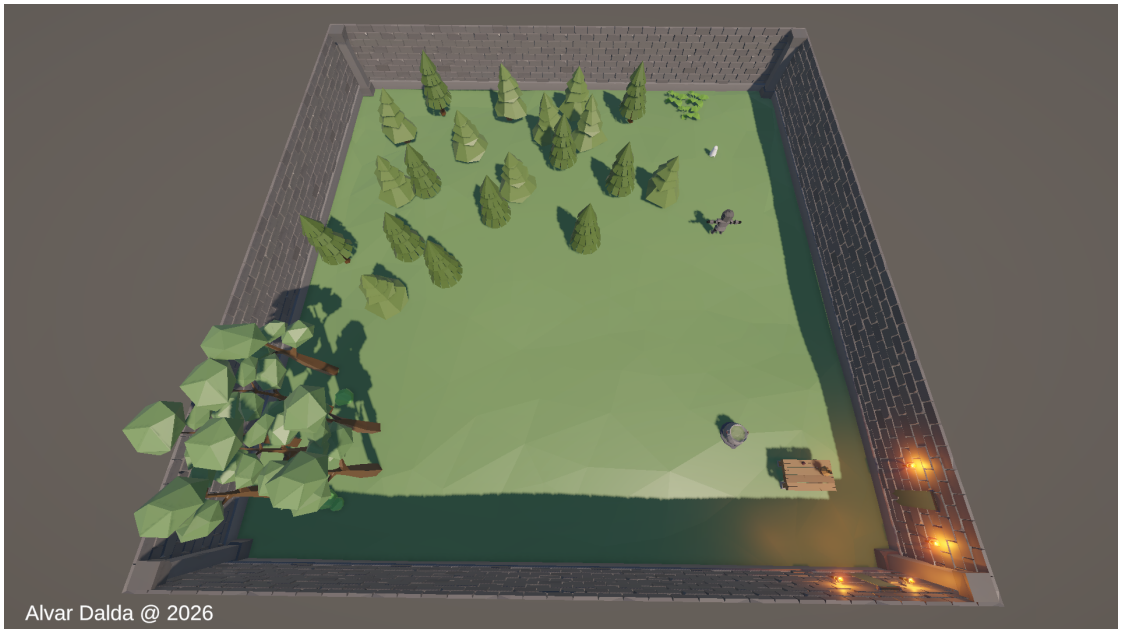


Figura 1: Vista general del jardín / patio del castillo.

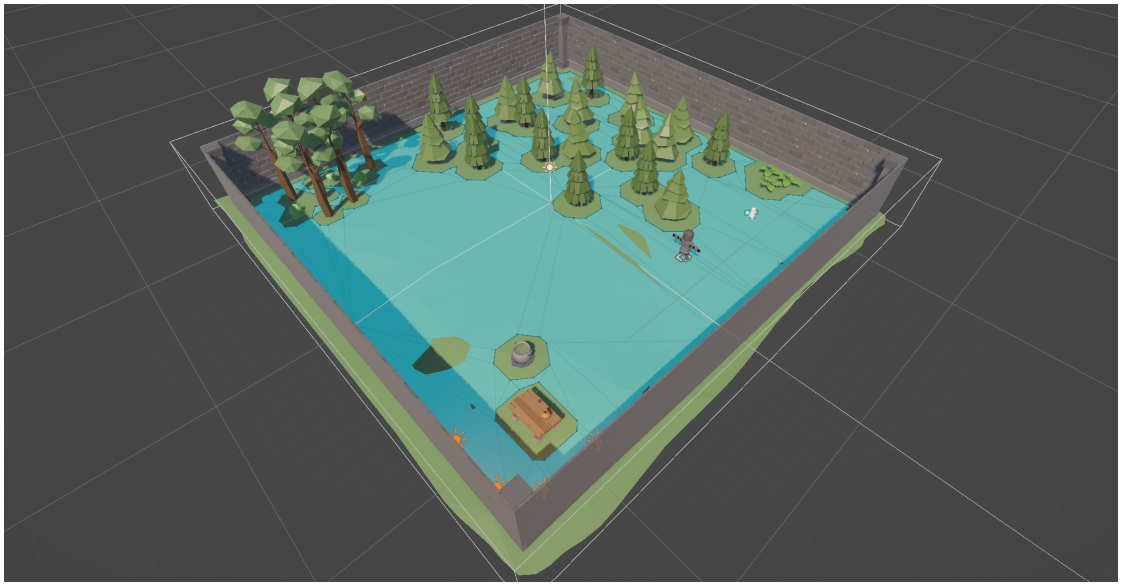


Figura 2: NavMesh tras el bake: zona caminable (azul) y huecos de los obstáculos.

3. Actividad 2: agente fantasma (*wander*)

3.1. Objetivo y restricciones

El enunciado pide un fantasma que deambule con *wander*, con movimiento creíble, y **sin C#**: solo Script Graph / Script Machine [4].

Uso el modelo Little Ghost [6] con un `NavMeshAgent`.

3.2. Algoritmo

Sigo el apunte de wander del material del curso [1], en la línea de Reynolds de un paseo que no cambia de dirección a cada frame [2]:

1. En espacio **local** del fantasma: punto aleatorio en la circunferencia (`Normalize` tras poner Y a cero), por `radius`, más un `(0,0,offset)` hacia delante.
2. Paso a mundo con `TransformPoint`.
3. `NavMesh.SamplePosition` para quedarme con un punto caminable; destino = `hit.position`.
4. Solo recalculo al llegar (`!pathPending` y `remainingDistance <arriveDistance`), no en cada frame.

Si el Sample falla (el punto cae fuera de la malla), un `For` (por ejemplo de 0 a 9) vuelve a probar hasta que hay hit y entonces `Break`. Si no sale nada, se espera a la siguiente llegada. Así no hay bucle infinito ni destinos imposibles.

El `offset` empuja el paseo hacia delante; el radio marca cuánto puede torcer. La velocidad del agente la dejé moderada para que se vea como deambular.

3.3. Implementación

Todo está en el Script Graph `GhostWander` del Script Machine. Variables principales: `radius`, `offset`, `arriveDistance`.

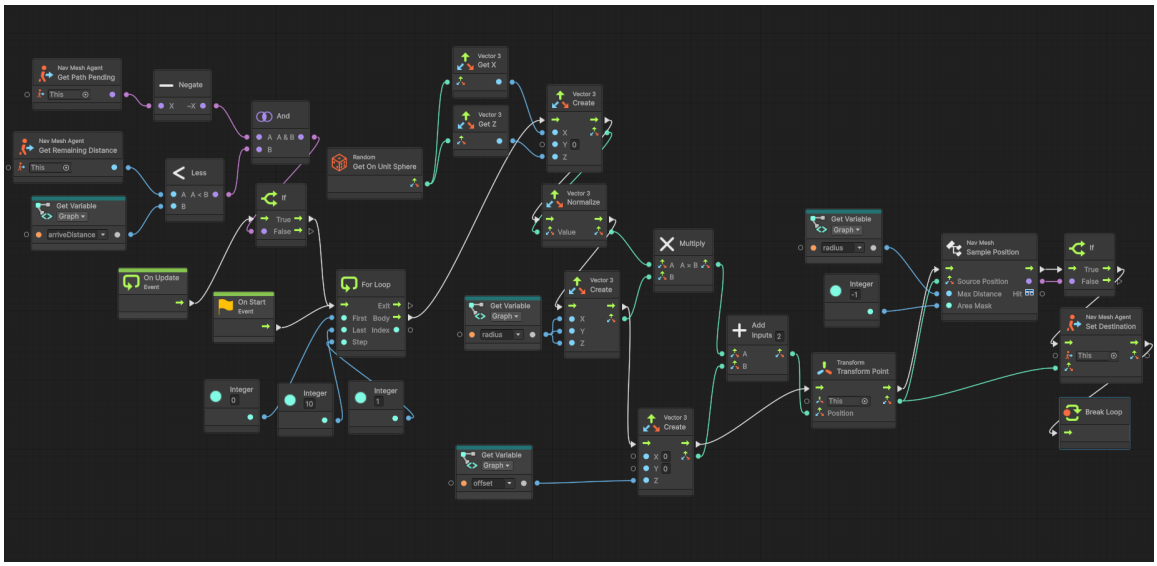


Figura 3: Script Graph del fantasma: wander, SamplePosition y SetDestination.

3.4. Vídeo

En la entrega y en el repo va un **vídeo de la actividad 2** (`ghost_wander.mp4`): el fantasma deambulando por la NavMesh, rodeando obstáculos, sin ruta fija de waypoints.

4. Actividad 3: agente de seguridad (máquina de estados)

4.1. Objetivo

Máquina de estados en Visual Scripting para un vigilante, con al menos:

- **Patrolling:** seguir una ruta por el escenario.
- **Flee:** alejarse del fantasma hasta poner distancia de por medio.

El vigilante es el Knight del pack Fantasy Heroes [7], con `NavMeshAgent` y un **State Machine** (`Assets/Scripts/Knight.asset`).

4.2. Esquema de la máquina de estados

Dos estados (`FlowState`): `Patroll` (inicial) y `Flee`. Las transiciones son grafos que llaman a `TriggerStateTransition` según la distancia al fantasma.

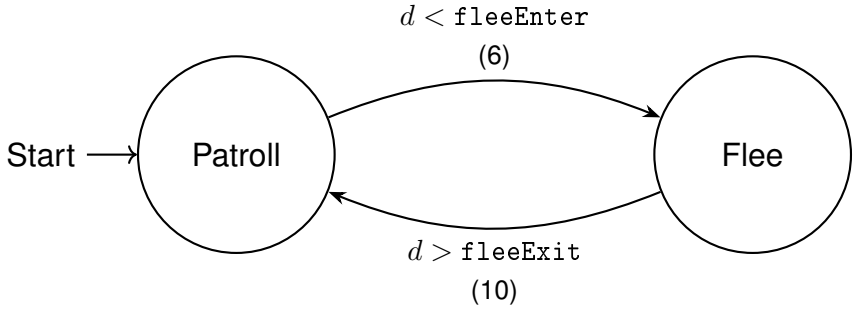


Figura 4: Máquina de estados del vigilante. La distancia para volver a patrullar (10) es mayor que la de entrar en huida (6), para que no esté cambiando de estado sin parar en el borde.

4.3. Estado Patroll

- Lista de waypoints (`points`) e índice `i`.
- Al entrar o al pasar de punto: `SetDestination` al waypoint actual.
- En `Update`: si no hay path pendiente y `remainingDistance <arriveDistance` (0,5), sube `i` (o vuelve al primero al acabar la lista).
- Velocidad de patrulla $\approx 2{,}7$.

4.4. Estado Flee

- Dirección de huida: $\mathrm{normalize}(\mathbf{p}_{\mathrm{knight}} - \mathbf{p}_{\mathrm{ghost}})$.
- Candidato: $\mathbf{p}_{\mathrm{knight}} + \mathrm{dir} \cdot \texttt{fleeDistance}$ ($\texttt{fleeDistance} = 8$), con Y a cero.
- $\texttt{NavMesh.SamplePosition}$ ($\texttt{maxDistance} = 20$, $\texttt{areaMask} = -1$); destino $= \texttt{hit.position}$.
- Velocidad de huida ≈ 5.

4.5. Transiciones

- **Patroll hacia Flee:** distancia al fantasma menor que `fleeEnterDistance` (6).
- **Flee hacia Patroll:** distancia mayor que `fleeExitDistance` (10).

Usar dos umbrales distintos evita el parpadeo de estado cuando el fantasma está justo en el límite. En una primera versión los `If` de las dos transiciones estaban mal cableados y el vigilante acababa *persiguiendo* al fantasma; cómo se corrigió está en la sección de IA generativa.

4.6. Animación de caminar

El pack del Knight [7] trae el modelo y el metarig, pero **no** un clip de walk usable. La animación de caminar del proyecto la generé con ayuda de IA (clip, Animator Controller y prefab); no venía en el recurso. El detalle está en la sección 7.3.

4.7. Evidencias visuales

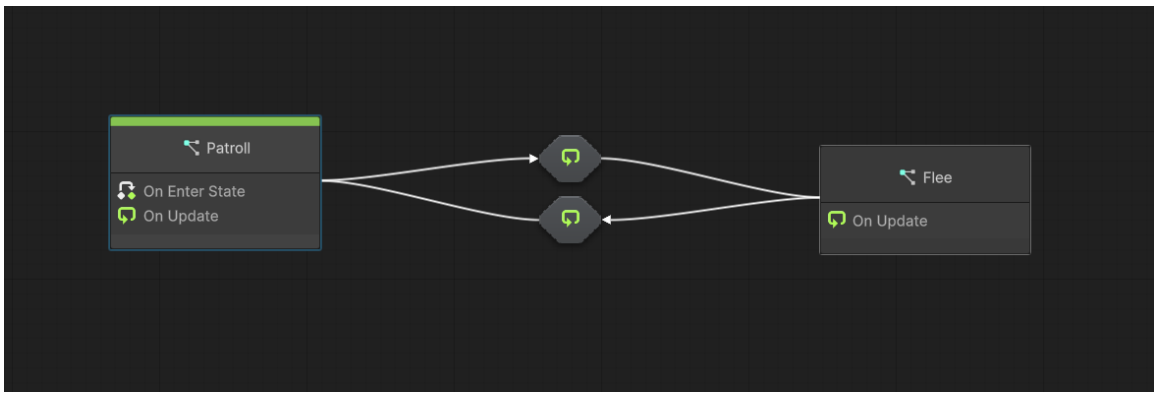


Figura 5: State Machine del Knight: Patroll, Flee y transiciones.

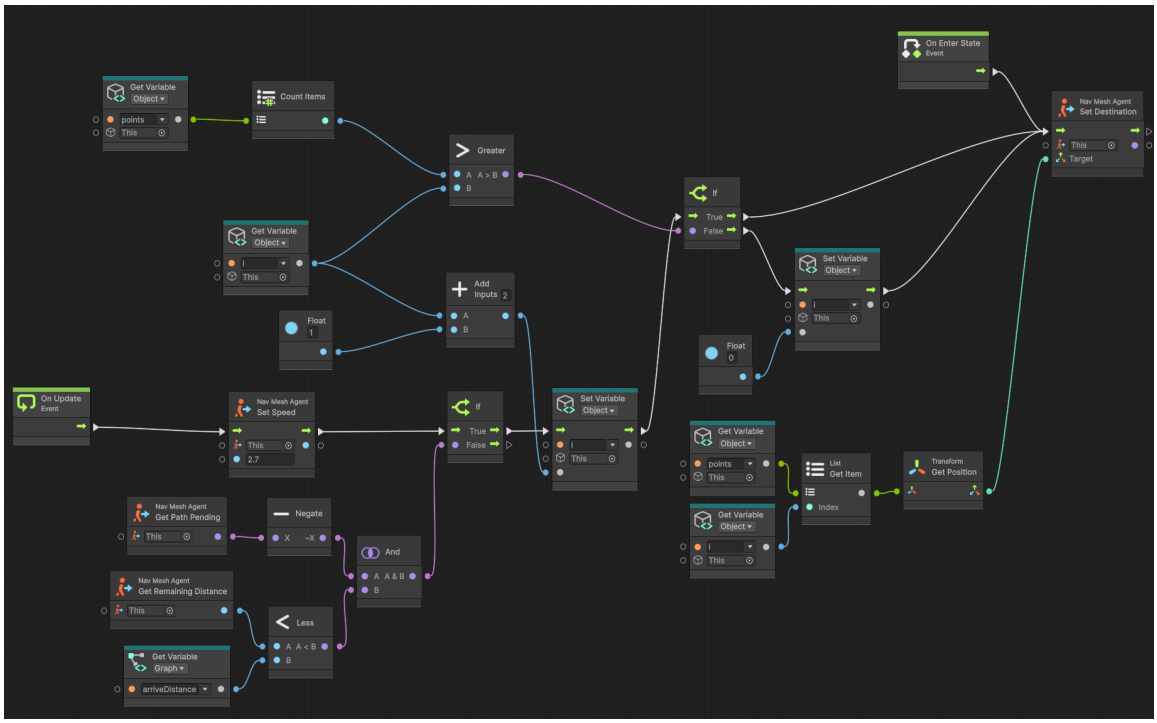


Figura 6: Grafo del estado Patroll (waypoints e índice).

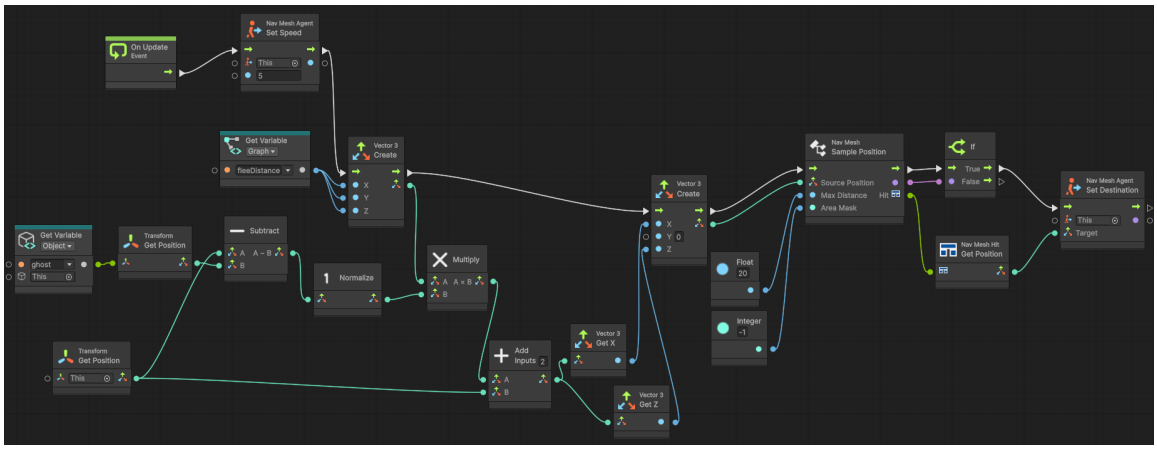


Figura 7: Grafo del estado Flee (dirección de alejamiento, SamplePosition y destino).

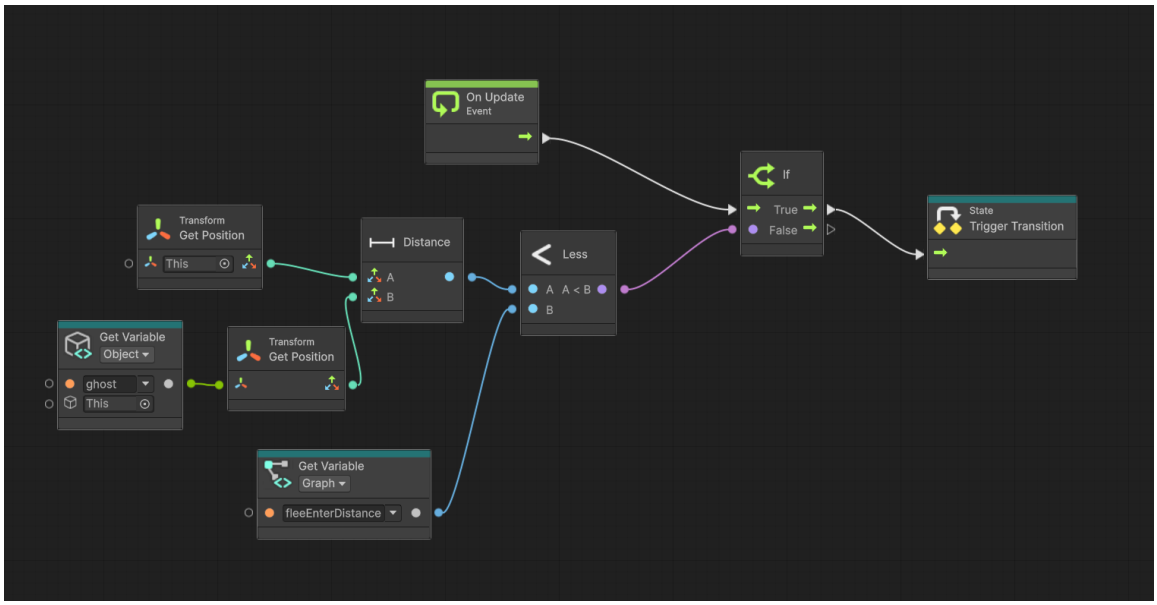


Figura 8: Transición Patroll hacia Flee: si la distancia es menor que `fleeEnterDistance`, `TriggerStateTransition`.

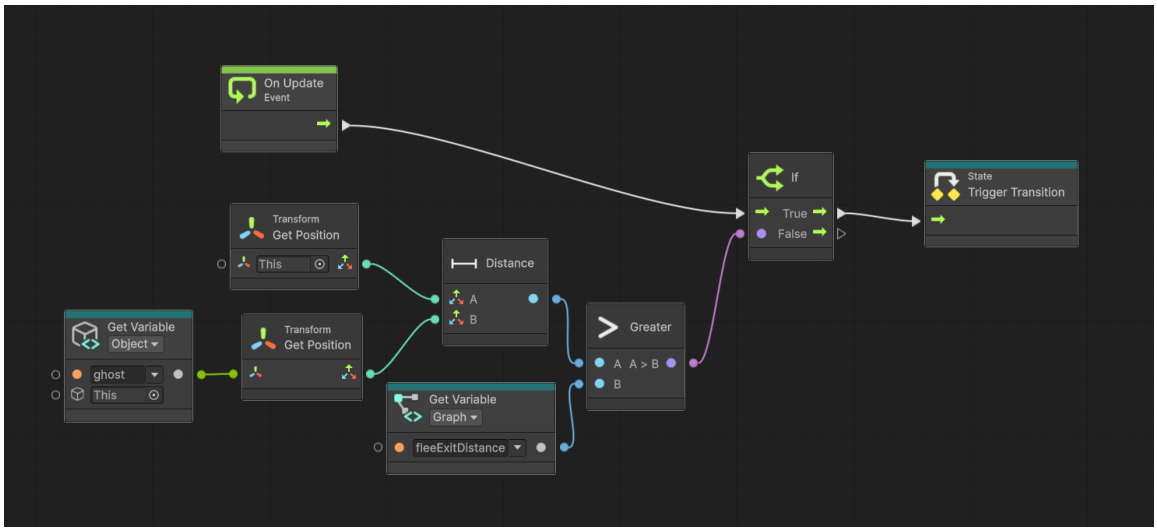


Figura 9: Transición Flee hacia Patroll: si la distancia es mayor que `fleeExitDistance`, `TriggerStateTransition`.

4.8. Vídeo

En la entrega y en el repo va un **vídeo de la actividad 3** (`knight_patroll+flee.mp4`): la ronda del vigilante, la huida cuando se acerca el fantasma y la vuelta a patrullar al alejarse.

5. Controles de demo

Para moverme por el escenario y grabar las demos tengo el script C# `CameraPOV` (`Assets/Scripts/CameraPOV.cs`), con el Input System de Unity:

- Ratón: yaw / pitch (POV).
- WASD: movimiento en el plano XZ (W delante según a dónde mira la cámara en horizontal).
- Espacio / Shift: subir / bajar.
- Escape: cierra la app (en el Editor para el Play).

No lo pide el enunciado de agentes, pero ayuda mucho a mirar la escena y a grabar los vídeos.

6. Recursos ajenos

He usado assets de terceros. Abajo van nombre, origen y uso. Las licencias son las de itch.io / Asset Store en el momento de la descarga; el uso es académico, para esta práctica.

Recurso	Origen	Uso en la PEC
Low Poly Fantasy Heroes (Knight) [7]	itch.io	Modelo del vigilante
Little Ghost lowpoly [6]	Asset Store	Modelo del fantasma
Low-Poly Simple Nature Pack [8]	Asset Store	Árboles / naturaleza
Low Poly Dungeons Lite [9]	Asset Store	Muros / patio

Cuadro 1: Recursos externos usados en la PEC1.

Nota sobre el Knight. El pack [7] da el personaje low-poly y el esqueleto (metarig), pero **no** traía una walk lista para el `NavMeshAgent`. La del proyecto salió de un prompt a un asistente de IA (sección 7.3).

7. IA generativa

He usado IA generativa como apoyo. Aquí van tres casos, con contexto, prompt y valoración.

7.1. Corrección de la lógica de huida del vigilante

Contexto. El State Machine del vigilante ya estaba montado, pero en Play el agente iba mal (a veces perseguía al fantasma). Le pasé a la IA el PDF de la práctica y el fichero de Visual Scripting para que me dijera qué fallaba.

Prompt (estilo de la petición).

> *Aquí tienes el PDF de la práctica y aquí el fichero de Visual Scripting. ¿Puedes comprobarme y decirme qué está fallando, para que lo pueda corregir?*

Respuesta (síntesis). La fórmula de huida estaba bien; fallaban los dos `If` de las transiciones (condiciones o ramas True/False mal puestas). Al corregirlos, entrar y salir de Flee cuadraba con los umbrales 6 y 10.

Valoración. Útil para pillar un error de cableado sin rehacer el algoritmo.

7.2. Creación del script de cámara (`CameraPOV`)

Contexto. Necesitaba moverme en primera persona por el patio para probar agentes y grabar. Pedí el script en C# sobre un fichero que ya había creado en el proyecto.

Prompt (consolidado, estilo de la petición original).

> *También en CameraPOV hazme el movimiento. Toma 3 inputs: la orientación del ratón, la W y la S. La orientación sirve para dirigir dónde mira la cámara, sencillo, un POV en primera persona. La W va hacia delante y la S hacia detrás, pero tiene que ser la Z local de la cámara: si miro hacia abajo y aprieto W, voy hacia abajo, hacia donde mira la cámara. Ya te he creado el script en Assets/Scripts/CameraPOV.cs.*
>
> *Okay, no me gusta del todo: añade también A y D; estas siempre me moverán en el plano XZ, ¿no?*
>
> *Vale, cambio: WASD las 4 direcciones en el plano XZ, W siendo delante siempre (igual que ahora pero sin subir y bajar), y para subir ESPACIO y bajar SHIFT.*
>
> *También en CameraPOV hazme que el botón ESC sea para cerrar la app.*

Resultado. Script con Input System: ratón (yaw/pitch), WASD en XZ con `forward`/`right` en el plano horizontal, Space/Shift en Y, Escape con `Application.Quit` (y para el Play en el Editor).

Valoración. Muy útil: fui afinando el control a base de mensajes cortos sin escribir cada variante a mano. Quedó tal como lo necesitaba para grabar.

7.3. Animación de caminar del Knight

Contexto. El Knight del pack [7] no traía walk. Con la patrulla ya montada, probé si la IA podía sacar el ciclo de caminar, el Animator y el prefab a partir del FBX.

Conversación (extracto).

> *Crees que podrías crearme de cero una animación para el recurso Assets/-Fantasy_Heroes_Free/FBX/Characters/Knight.fbx? Solo necesitaría una animación para caminar, pero eso requiere ponerle un esqueleto, crear la animación, crear un prefab con el animator, etc. No sé si tienes la capacidad de hacerlo pero tengo curiosidad a ver si puedes.*

La respuesta vio que el FBX **ya** traía metarig de Blender y skinning, pero Unity lo importaba sin avatar ni clips. Por código se generó un ciclo en bucle ($\approx 0{,}7$ s):

- `Assets/Fantasy_Heroes_Free/Animations/Knight_Walk.anim`
- `Assets/Fantasy_Heroes_Free/Animations/Knight.controller`
- `Assets/Fantasy_Heroes_Free/Prefabs/Knight.prefab`
- Menú para regenerar: **Tools** / **Build Knight Walk** (`KnightWalkBuilder.cs`)

Proceso. Revisión del FBX, reimport con avatar, poses procedurales (16 muestras), ajuste de piernas/brazos y del pie de apoyo, y asignación al Knight de la escena sin root motion (para no pelearse con el `NavMeshAgent`).

Valoración. Cubrió el hueco del asset (modelo sí, walk no). Limitaciones: el ciclo es procedural (no mocap); el paso va a $\approx 0{,}9$ m/s y la patrulla a $\approx 2{,}7$ m/s, así que hay algo de deslizamiento de pies si no tocas el Speed del Animator o del Agent. Quede claro: la IA no modeló el personaje; solo el clip y el Animator.

8. Conclusiones

Queda un patio con NavMesh, un fantasma con wander en Script Graph, un vigilante Patroll/Flee en State Machine y una cámara para demos. Lo más delicado ha sido afinar el wander (parámetros y SamplePosition), los dos umbrales de las transiciones de huida y la walk del Knight, que no venía en el pack y salió con IA. Los vídeos de las actividades 2 y 3 van con este informe.

La base sirve para meter más agentes en las PEC siguientes sin rehacer el escenario.

Bibliografía

[1] Gerard Escudero. *Game AI en Unity*. URL: https://github.com/gebakx/gameAIUnity/tree/main (visitado 01-11-2026).

[2] Craig W. Reynolds. *Steering Behaviors For Autonomous Characters*. URL: https://www.red3d.com/cwr/steer/gdc99/ (visitado 01-11-2026).

[3] Unity Technologies. *Inner Workings of the Navigation System*. URL: https://docs.unity3d.com/Packages/com.unity.ai.navigation@1.1/manual/NavInnerWorkings.html (visitado 01-11-2026).

[4] Unity Technologies. *Visual Scripting Manual*. URL: https://docs.unity3d.com/Packages/com.unity.visualscripting@1.9/manual/index.html (visitado 01-11-2026).

[5] Alvar Dalda Svärd. *IAV_R1: PEC1 - Inteligencia artificial para videojuegos*. URL: https://github.com/Raptor200111/IAV_R1 (visitado 02-11-2026).

[6] Unity Asset Store. *Little Ghost lowpoly (Free)*. URL: https://assetstore.unity.com/packages/3d/characters/little-ghost-lowpoly-free-271926 (visitado 01-11-2026).

[7] StandOut7. *Low Poly Fantasy Heroes*. URL: https://standout7.itch.io/fantasy-heroes (visitado 01-11-2026).

[8] Unity Asset Store. *Low-Poly Simple Nature Pack*. URL: https://assetstore.unity.com/packages/3d/environments/landscapes/low-poly-simple-nature-pack-162153 (visitado 01-11-2026).

[9] Unity Asset Store. *Low Poly Dungeons Lite*. URL: https://assetstore.unity.com/packages/3d/environments/dungeons/low-poly-dungeons-lite-177937 (visitado 01-11-2026).